

计算机网络实践说明书

TCP 在应用层的设计与实现

2026 年 8 月修订

1. 简介

本次实践要求学生在应用层使用 UDP 实现 TCP 的基本功能。学生依据 TCP 相关标准，实现标准 TCP 中 `listen`、`connect`、`accept`、`send`、`recv` 和 `close` 等接口，以及连接管理、可靠数据传输、流量控制和基础拥塞控制等核心功能；完整快速恢复及现代拥塞控制与丢包恢复机制作为挑战任务。

为区别于操作系统内核中的标准 TCP 实现，本文将学生实现的协议称为 TJU_TCP。TJU_TCP 是基于 TCP 基本原理设计的教学用简化可靠传输协议，采用自定义报文头并承载在 UDP 之上，不与互联网标准 TCP 直接互操作。

实践采用 Linux 操作系统和 C 语言。学生在教师提供的编程框架基础上进行开发，通过两个相互通信的运行环境分别模拟服务端和客户端，并使用 UDP 传输 TJU_TCP 的控制报文和数据报文。借助网络环境配置工具可以模拟丢包、时延和带宽限制；借助 Wireshark、tcpdump 及课程提供的日志和绘图工具，可以观察协议在不同网络条件下的运行过程。

本实践采用“个人项目制+AI 辅助开发”模式。每名学生独立完成代码框架分析、协议设计、编程实现、功能测试、性能评价、课程报告和个人答辩。AI 编程工具和 AI 写作工具可以辅助完成代码理解、架构分析、方案检查、代码生成、测试设计、错误诊断和文档优化，但学生必须核验 AI 输出，并对提交的全部代码、文档和实验结果承担责任。

为保证评价公平，本实践采用“阶段性提交+自动测试+过程材料+个人答辩”相结合的验收方式。自动测试主要评价协议功能，过程材料和答辩主要评价学生对框架、协议、代码和实验结果的理解，以及其使用和验证 AI 工具的能力。

1.1 实践目标

通过本次实践，学生应能够：

1. 理解连接管理、可靠数据传输、流量控制和拥塞控制的基本原理；
2. 阅读并使用 Internet 协议标准，将协议要求转化为可实现、可测试的软件需求；
3. 分析已有编程框架，识别模块、接口、控制流、数据流、关键数据结构及可修改边界；
4. 在不可靠网络环境中设计并实现一个具有连接管理、可靠传输、流量控制和基础拥塞控制能力的简化传输协议；
5. 使用抓包、日志、自动测试和性能实验验证协议实现的正确性；
6. 使用 AI 工具辅助代码理解、设计、开发、测试和文档撰写，并能够识别、修正和验证 AI 输出；
7. 通过技术报告和个人答辩准确说明设计依据、实现方法、实验结果与局限性。

1.2 基本原则

1. **个人负责：**每名学生独立完成全部必做任务，使用独立代码仓库并单独提交。

2. **标准驱动**：协议设计以课程指定的 RFC 和任务要求为依据。
3. **框架约束**：必须在教师提供的框架和规定接口内实现，不得绕过接口另起炉灶。
4. **测试驱动**：每一阶段均应先明确验收条件，再完成实现和验证。
5. **人机协同**：AI 可以提出分析或实现建议，但重要结论必须有源代码、标准、编译运行、日志、抓包或测试结果作为证据。
6. **过程可追溯**：使用 Git 保存阶段性版本，保留必要的 AI 协作记录、测试记录和实验数据。

1.3 主要成果

每名学生应提交：

1. TJU_TCP 完整源代码；
2. Git 提交记录；
3. 第 1 周、第 2 周和第 3 周周末提交的三次阶段进度报告；
4. 最终课程报告，包括功能测试、性能测试的原始数据、图表和分析；AI 使用说明和代表性 AI 协作记录等。
5. 个人展示材料及按要求录制的视频；
6. 教师或助教要求的其他验收材料。

2. 标准依据与任务范围

2.1 主要标准

本实践主要参考以下标准：

1. RFC 9293, *Transmission Control Protocol (TCP)*: TCP 基本规范，是 RFC 793 的替代标准；
2. RFC 6298, *Computing TCP's Retransmission Timer*: RTT 估计和重传超时计算；
3. RFC 5681, *TCP Congestion Control*: Reno 慢启动、拥塞避免、快速重传和快速恢复；
4. RFC 2018, *TCP Selective Acknowledgment Options*: TCP 选择确认选项；
5. RFC 6582, *The NewReno Modification to TCP's Fast Recovery Algorithm*: NewReno 快速恢复；
6. RFC 6675, *A Conservative Loss Recovery Algorithm Based on Selective Acknowledgment (SACK) for TCP*: 基于 SACK 的丢包恢复；
7. RFC 7323, *TCP Extensions for High Performance*: 窗口扩展和时间戳选项；
8. RFC 8985, *The RACK-TLP Loss Detection Algorithm for TCP*: 基于时间和确认信息的现代丢包检测；
9. RFC 9438, *CUBIC for Fast and Long-Distance Networks*: CUBIC 拥塞控制；
10. RFC 793, *Transmission Control Protocol*: 作为 TCP 经典版本和教学历史对照。

课程只选取上述标准中的部分功能形成简化的 TJU_TCP。RFC 9293、RFC 6298 和 RFC 5681 用于必做功能的主要标准依据；其余现代扩展主要用于挑战任务和技术对照。说明书明确标注为“教学简化”或“不要求实现”的内容不参与必做功能验收，但学生应了解这些裁剪与完整标准 TCP 之间的差异。

2.2 必做与选做任务

类型	内容
必做	编程环境搭建与基线运行
必做	已有编程框架的代码架构分析
必做	任务分析和协议总体设计
必做	连接建立与连接关闭
必做	可靠数据传输
必做	流量控制
必做	基础 Reno 拥塞控制：慢启动、拥塞避免和丢包后的窗口降低
必做	功能测试、联调和性能评价
必做	课程报告、AI 使用说明和答辩材料
挑战任务 (选做)	完整 Reno 快速恢复、NewReno、SACK/RACK 或 CUBIC

3. 三阶段实践任务

课程按周组织为三个阶段，每周完成一个阶段，并在该周周末提交阶段成果。

阶段	时间	任务	阶段提交时间
第一阶段： 环境、框架分析与总体设计	第 1 周	环境搭建与基线运行； 代码架构分析与恢复； 任务分析与协议总体设计	第 1 周周末
第二阶段： 协议核心功能实现	第 2 周	连接管理；可靠数据传输与流量控制	第 2 周周末
第三阶段： 基础拥塞控制、综合测试与 成果提交	第 3 周	基础 Reno 拥塞控制； 联调、功能测试与性能评价； 最终报告、代码复现与个人答辩	第 3 周周末
挑战任务（选做）： 高级拥塞控制与丢包恢复	完成必 做任务 后	完整 Reno 快速恢复、NewReno、SACK/RACK、CUBIC， 可选择一项或多项	第 3 周周末

3.1 第一阶段：环境、框架分析与总体设计（第 1 周）

3.1.1 子任务一：环境搭建与基线运行

学生应完成以下工作：

1. 安装或导入课程指定的 Linux 运行环境；
2. 获取教师发布的编程框架，核对框架版本；
3. 完成编译并运行课程提供的基线程序；
4. 验证服务端和客户端之间的 UDP 通信；
5. 建立个人 Git 仓库，提交未经修改的基线版本；
6. 记录操作系统、编译器、依赖、框架版本和测试脚本版本；
7. 验证网络参数配置、抓包和日志工具能够正常使用。

子任务成果：环境配置记录、基线运行结果、框架版本标识和初始 Git 提交。

3.1.2 子任务二：代码架构分析

学生应使用 AI 编程工具辅助分析教师提供的已有框架，但不得直接把 AI 输出作为最终结论。分析内容至少包括：

1. 项目目录结构及主要文件职责；
2. 程序启动入口、构建过程和运行方式；
3. TJU_TCP 八个规定接口之间的调用关系；
4. 服务端和客户端的主要执行流程；
5. 控制报文和数据报文的发送、接收与处理路径；
6. tju_tcp_t、TJU_TCP 报文和缓冲区等关键数据结构；
7. 状态机、线程、锁、条件变量、定时器等机制在框架中的位置；
8. 框架已经实现的功能、需要学生实现的功能和限制修改的内容；
9. 可能影响可靠性、并发安全和性能的关键点；
10. AI 分析中存在的不确定结论及其验证方法。

子任务成果至少包括：

- （1）一张模块架构图；
- （2）一张主要函数调用或数据流图；
- （3）关键数据结构和接口说明；
- （4）至少三项通过代码定位、编译运行、日志或测试验证的架构分析结论；
- （5）至少一项对 AI 错误判断或不完整结论的修正记录。

3.1.3 子任务三：任务分析与协议总体设计

学生应将本说明书的功能要求转化为设计方案和测试条件，至少完成：

1. 需求分解和必做功能清单；
2. “任务要求—标准依据—设计模块—测试方法”的追踪关系；
3. TJU_TCP 总体架构及模块接口；
4. 连接管理、可靠传输和流量控制的状态或流程设计；
5. 报文格式、关键数据结构和缓冲区管理方案；
6. 并发控制、定时器和异常处理方案；
7. 分阶段实现计划及风险分析。

AI 工具可用于提取需求、检查设计完整性和发现边界情况。学生必须对照本说明书、RFC 和框架代码核验 AI 建议，并在设计文档中保留最终采用方案及其理由。

第一阶段成果应在第 1 周周末统一提交，包括三个子任务的全部材料、对应 Git 版本和代表性 AI 协作记录。

3.2 第二阶段：协议核心功能实现（第 2 周）

3.2.1 子任务一：连接管理

完成连接建立、连接关闭及相关异常处理，实现第 5.1 节和第 5.2 节规定的功能，并通过本地测试和线上验收。

3.2.2 子任务二：可靠数据传输与流量控制

完成发送端和接收端缓冲区、滑动窗口、序号与确认号管理、校验和、累计确认、丢包检测、快速重传、超时重传、RTT/RTO 估计以及流量控制，实现第 5.3 节和第 5.4 节规定的功能。

第二阶段成果应在第 2 周周末统一提交，包括两个子任务的源代码、Git 版本、自动测试结果、日志或抓包、进度报告和代表性 AI 协作记录。

3.3 第三阶段：基础拥塞控制、综合测试与成果提交（第 3 周）

3.3.1 子任务一：基础 Reno 拥塞控制

依据第 5.5 节实现基础 Reno 拥塞控制，至少包括拥塞窗口 `cwnd` 和慢启动阈值 `ssthresh` 的维护、慢启动、拥塞避免，以及检测到超时或三次重复 ACK 所表示的丢包后降低发送窗口。发送端实际允许的在途数据量同时受接收窗口 `rwnd` 和拥塞窗口 `cwnd` 约束。

学生应通过 trace、日志或抓包展示 `cwnd`、`ssthresh`、`rwnd`、在途数据量和丢包事件之间的关系，并说明慢启动、拥塞避免和窗口降低的触发条件。完整快速恢复不属于本子任务的必做内容。

3.3.2 子任务二：联调、功能测试与性能评价

在多种丢包率、时延、带宽、缓冲区和报文长度条件下测试 TJU_TCP，记录原始数据，绘制关键指标随时间或网络条件变化的图表，解释实验现象，分析协议实现的局限性并提出改进方法。

3.3.3 子任务三：最终报告、代码复现与个人答辩

整理并提交最终源代码、构建与运行说明、原始实验数据、绘图脚本、课程报告、AI 使用声明和个人展示材料；复现主要功能与性能实验，并参加个人答辩及必要的代码讲解或现场修改。

第三阶段成果应在第 3 周周末统一提交，包括基础 Reno 源代码和测试证据、综合实验原始数据与图表、最终代码、课程报告、AI 使用说明和个人展示材料。个人答辩的具体时间和形式以课程通知为准。

3.4 挑战任务：高级拥塞控制与丢包恢复

在完成所有必做任务后，可以按第 5.6 节选择实现一项或多项高级机制：完整 Reno 快速恢复、NewReno、SACK 及其丢包恢复、RACK-TLP 或 CUBIC。挑战任务不影响必做任务的基本分值。不同挑战方向的难度和加分标准由课程平台另行发布。

4. TJU_TCP 报文格式

TJU_TCP 采用课程自定义的简化报文头，头部固定为 20 字节。该格式借鉴 TCP 的部分字段，但不是 RFC 9293 规定的标准 TCP 报文头，不包含标准 TCP 选项空间。字段如下：

字段	建议长度	说明
Source Port Number	16 bit	源端口号
Destination Port Number	16 bit	目的端口号
Sequence Number	32 bit	序列号
Acknowledgement Number	32 bit	确认号
Header Length	16 bit	报文头长度，TJU_TCP 中固定为 20 字节
Packet Length	16 bit	整个报文长度，包括报文头和数据
Flags	8 bit	SYN、ACK、FIN 等控制标志
Advertised Window	16 bit	接收方从确认号所指字节开始愿意继续接收的数据字节数，用于流量控制
Extension	8 bit	保留字段，用于使报文头长度为 20 字节

若框架中的报文结构与本表存在差异，以课程平台发布的当前版本和补充说明为准。

Advertised Window 为 16 位无符号数。未实现 RFC 7323 窗口扩展时，通告值不得超过 65535 字节；当实际接收缓冲区可用空间更大时，应通告 $\min(\text{可用空间}, 65535)$ ，不得通过整数截断造成窗口回绕。

5. TJU_TCP 功能要求

本章是功能验收和评分的重要依据。课程报告的协议设计、协议实现和实验验证必须覆盖全部必做条目。下列要求在 RFC 9293、RFC 6298 和 RFC 5681 的基础上，结合三周个人实践的工作量进行了裁剪；未纳入的标准 TCP 强制功能均以“教学简化”明确标注。

5.1 连接建立

1. 实现基本三次握手；
2. 正确生成并处理 SYN、SYN-ACK 和 ACK 报文，SYN 占用一个序列号，ACK 确认对方下一个期望的序列号；
3. 正确维护客户端和服务端的连接状态；
4. 为不同连接选择变化的 32 位初始序列号 ISN，不得对所有连接固定使用 0 或其他常量；推荐采用单调变化的时钟分量，并加入连接四元组和本地产生的不可预测分量；
5. 能够在存在丢包和时延的条件下建立连接；
6. 能够处理三次握手中不同报文丢失所引起的重传，包括对重复 SYN-ACK 重新发送有效 ACK；
7. 握手报文的重传定时遵守第 5.3 节的 RTO 规则；若使用小于 3 秒的初始 RTO 且 SYN 发生重传，连接建立后的数据阶段应将 RTO 重新初始化为 3 秒；
8. tju_connect、tju_listen 和 tju_accept 的行为满足接口规范。

为教学简单起见，以下内容不要求实现：

- (1) 同时建立连接（simultaneous open）；
- (2) 旧重复 SYN（Old Duplicate SYN）处理；
- (3) 建立连接期间的双方崩溃（Crash）；
- (4) 半开放连接发现（Half-Open Connection Discovery）；
- (5) 标准 TCP 的 RST 机制。

5.2 连接关闭

1. 实现双向 FIN 关闭过程；典型情况下使用四个报文完成关闭，但 ACK 可以与本端 FIN 合并；
2. 能够在双方先后关闭连接时正确完成关闭；
3. 能够在双方同时关闭连接时正确处理对方的 FIN 及相关 ACK；
4. 能够在存在丢包和时延的条件下关闭连接；

5. 调用 `tju_close` 后不再接受新的发送请求；此前已经提交的发送数据排列在 `FIN` 之前，并与 `FIN` 一起持续重传，直至得到确认或连接因超时而异常终止；
6. 连接关闭过程不得丢弃、覆盖或破坏尚未完成的数据传输；
7. 正确维护 `FIN-WAIT`、`CLOSE-WAIT`、`CLOSING`、`LAST-ACK` 和 `TIME-WAIT` 等关闭状态及相关定时行为；
8. 主动关闭方进入 `TIME-WAIT` 后应等待 $2 \times \text{TJU_MSL}$ 再释放连接状态；课程可为自动验收设置缩短的 `TJU_MSL`，具体值以课程平台为准。

以下内容不要求实现：

- (1) 关闭连接的过程中双方崩溃；
- (2) 一方直接退出且不响应对方 `FIN` 的完整故障恢复，但必须实现 `FIN` 超时重传和课程规定的最终终止条件；
- (3) `SO_LINGER` 机制；
- (4) 标准 TCP 的 `RST` 机制(比如在接收缓冲区存在数据时关闭 `socket` 不需要发送 `RST`，已经关闭的 `socket` 接收到数据不需要发送 `RST`)

5.3 可靠数据传输

1. 正确管理发送缓冲区并实现发送滑动窗口；
2. 正确管理接收缓冲区并实现接收滑动窗口；
3. 正确管理 32 位字节序列号和确认号，正确处理 `SYN`、`FIN` 占用的序列号以及序列号回绕比较；
4. 按第 4 章规定计算并校验 `checksum`，能够检测报文差错；校验失败的报文不得作为有效数据或有效确认继续处理；
5. 实现累计确认，确认号表示接收端下一个期望接收的字节序列号；
6. 检测并处理报文丢失、重复、重叠和失序，只向应用按序交付一次数据；
7. 实现基于三次重复 `ACK` 的基本快速重传；
8. 依据 RFC 6298 维护平滑往返时间 `SRTT`、往返时间偏差 `RTTVAR` 和重传超时 `RTO`：无 `RTT` 样本时初始 `RTO` 取 1 秒；取得首个样本 `R` 后设置 $\text{SRTT}=\text{R}$ 、 $\text{RTTVAR}=\text{R}/2$ 、 $\text{RTO}=\text{SRTT}+\max(\text{G}, 4 \times \text{RTTVAR})$ ；后续样本按 RFC 6298 规定更新；
9. 使用 Karn 算法消除重传歧义：除非使用时间戳等机制能够确定确认对应的发送实例，否则不得从已经重传的报文取得 `RTT` 样本；在可采样时，每个 `RTT` 至少取得一个 `RTT` 样本；
10. 根据 `RTO` 实现超时重传：超时时重传最早未确认报文，将 `RTO` 加倍并重新启动定时器；收到推进累计确认点的新 `ACK` 时重新启动定时器；全部在途数据确认后停止定时器；不得早于一个 `RTO` 重复重传同一报文；
11. 正确处理 `ACK` 推进、已确认数据释放和窗口滑动；
12. 在存在时延和丢包的 network 环境中，可靠传输不少于 100 MB 的数据；

13.tju_close 返回前，应按课程接口语义完成此前已提交发送数据的可靠交付，或者明确返回课程规定的失败状态，不得默认丢弃缓冲数据。

以下内容不要求实现：

- (1) TCP urgent 机制；
- (2) 传输过程中双方崩溃恢复；
- (3) 延迟确认；这是对 RFC 9293 推荐功能的裁剪；未实现时应及时发送 ACK
- (4) 为等待捎带确认而额外延迟 ACK；连接建立后的发送报文仍应携带当前有效确认号，在不引入不合理延时可以把确认与数据一起发送；
- (5) SACK 和 D-SACK；SACK 及其丢包恢复列为挑战任务。

5.4 流量控制

1. 接收方根据接收缓冲区占用情况计算可用空间；
2. 接收方在 Advertised Window 字段中通告从确认号开始能够继续接收的字节数；未实现窗口扩展时，通告值取 $\min(\text{可用空间}, 65535)$ ；
3. 发送方根据通告窗口调整有效发送窗口；发送方不得发送超过接收方可接收范围的数据；实现第 5.5 节后，实际允许的在途数据量不得超过 $\min(\text{rwnd}, \text{cwnd})$ ；
4. 正确处理通告窗口为 0 的情况；实现零窗口探测机制：发送方在零窗口持续一个 RTO 后发送首次探测，后续探测间隔指数增长；接收方收到探测后返回包含下一期望序列号和当前窗口的 ACK；只要对方持续响应探测，发送方不得仅因窗口持续为 0 而关闭连接；
5. 接收方不应主动把已通告窗口的右边界向左移动；发送方必须能够处理对方窗口缩小，在可用窗口为负值或 0 时停止发送新数据，但仍按规则处理旧的未确认数据；
6. 发送端和接收端均实现基本的 Silly Window Syndrome (SWS) 避免机制，避免持续发送或通告过小窗口；
7. 通过日志、trace 或抓包结果展示流量控制的运行过程。

不要求实现 Nagle 算法。该算法是 RFC 9293 的推荐功能；若学生自行实现，必须允许应用按连接关闭该算法。

5.5 基础 Reno 拥塞控制（必做）

依据 RFC 5681 实现基础 Reno 拥塞控制：

1. 每条连接维护以字节为单位的拥塞窗口 cwnd 和慢启动阈值 ssthresh；
2. 发送端的实际发送上限由 $\min(\text{cwnd}, \text{rwnd})$ 决定；
3. 当 $\text{cwnd} < \text{ssthresh}$ 时执行慢启动，对每个累计确认新数据的 ACK 增加 cwnd，每次增加量不超过一个 SMSS；
4. 当 $\text{cwnd} \geq \text{ssthresh}$ 时执行拥塞避免，使 cwnd 在无拥塞时约每个 RTT 增加一个 SMSS，且不得比 RFC 5681 规定的 Reno 增长更激进；

5. 发生 RTO 超时，设置 $ssthresh = \max(FlightSize/2, 2 \times SMSS)$ ，并把 cwnd 降低到不超过一个 SMSS，随后重新进入慢启动；
6. 收到第三个重复 ACK 时执行基本快速重传，设置 $ssthresh = \max(FlightSize/2, 2 \times SMSS)$ 并降低 cwnd；如果没有选择实现快速恢复挑战任务，可以在重传数据得到新 ACK 后令 $cwnd = ssthresh$ 并进入拥塞避免；
7. SYN 或 SYN-ACK 的确认不得增加 cwnd；初始拥塞窗口取值应符合课程平台发布的 RFC 5681 兼容配置；
8. 通过 trace 和图表展示 cwnd、ssthresh、rwnd、在途数据量和丢包事件随时间的变化，并解释不同阶段的窗口变化。

5.6 高级拥塞控制与丢包恢复（挑战任务，可选）

学生在完成第 5.5 节必做功能后，可以选择以下一项或多项挑战任务：

1. **完整 Reno 快速恢复**：依据 RFC 5681 实现第三个重复 ACK 触发后的拥塞窗口膨胀、重复 ACK 期间的新数据发送，以及恢复 ACK 到达后的窗口收缩；
2. **NewReno**：依据 RFC 6582 处理快速恢复期间的部分确认，在无 SACK 时提高单个窗口内多报文丢失的恢复能力；
3. **SACK 及 SACK 丢包恢复**：依据 RFC 2018 协商和处理 SACK 选项，维护记分板，并参考 RFC 6675 实现基于 SACK 的保守丢包恢复；
4. **RACK-TLP**：依据 RFC 8985，结合逐报文发送时间、SACK 和探测报文实现基于时间的丢包检测；该任务以前置实现 SACK 和必要的时间信息为条件；
5. **CUBIC**：依据 RFC 9438 实现 CUBIC 拥塞避免和拥塞响应，并说明其与 Reno 线性增长和窗口降低规则的差异；
6. 每项挑战任务均须提交设计依据、实现说明、独立测试、原始 trace、可复现图表及与基础 Reno 的对比分析。

6. 接口规范与修改限制

为统一标准，方便测试，提供了项目的基础代码，并规定了 TJU_TCP 需要实现的各个接口。学生需要按照接口的定义和功能描述实现具体的代码。测试时将编写代码调用学生封装的接口，模拟服务端和客户端的行为，进行自动测试和验收。

6.1 规定接口

学生必须实现或完善以下八个接口，不得修改函数签名（函数名、参数位置、参数数量和返回类型）：

1) `tju_tcp_t * tju_socket ()`

创建 TJU_TCP

2) `int tju_bind (tju_tcp_t *sock, tju_sock_addr bind_addr)`

绑定监听的地址

3) `int tju_listen (tju_tcp_t *sock)`

被动打开

4) `tju_tcp_t * tju_accept (tju_tcp_t *sock)`

接受连接

5) `int tju_connect (tju_tcp_t *sock, tju_sock_addr target_addr)`

连接到服务端

6) `int tju_send (tju_tcp_t *sock, const void *buffer, int len)`

发送数据

7) `int tju_recv (tju_tcp_t *sock, void *buffer, int len)`

接收数据

8) `int tju_close (tju_tcp_t *sock)`

关闭连接

具体输入、输出和功能说明以课程框架 `documentation` 目录中的接口文档为准。

6.2 接口约束

1. 不得修改规定接口的签名：函数名、参数顺序、参数数量和返回类型；
2. 不得以自定义接口替代规定接口；
3. 可以添加辅助函数、内部数据结构和源文件，但不得破坏自动测试的调用方式；
4. 所有接口必须能够在课程指定环境中编译、链接和运行；
5. AI 生成代码同样必须遵守上述约束。

6.3 限制修改内容

以下内容的修改必须事先向教师或助教申请并获得确认：

1. `kernel.h` 和 `kernel.c` 中除 `cal_hash` 外的函数和变量定义；
2. `tju_packet.h` 和 `tju_packet.c` 中除 `tju_packet_t` 报文结构外的其他结构和函数；
3. `global.h` 中 TCP socket 状态、最大包长、标志位和通用数据长度等全局定义；
4. 课程测试程序、验收脚本和构建流程中的关键检查逻辑；
5. 教师明确标注为只读或禁止修改的其他文件。

未经批准修改限制内容，或者通过固定结果、跳过处理、识别测试程序等方式绕过验收，视为无效实现。

7. 参数与全局约定

1. TJU_TCP 最大报文总长度设置为 1400 字节，以避免 IP 层分片；该值包括 20 字节 TJU_TCP 头部和有效数据，不再称为 TCP MSS；
2. 发送端最大报文数据长度 SMSS 为 $1400 - 20 = 1380$ 字节；本文涉及 cwnd、sssthresh 和拥塞窗口更新时，SMSS 均指不含 TJU_TCP 头部的数据字节数；
3. 发送缓冲区和接收缓冲区的容量均不得小于 $5000 \times \text{SMSS}$ 字节；
4. 未实现窗口扩展时，Advertised Window 最大通告值为 65535 字节；缓冲区容量大于该值不代表可以在 16 位字段中直接通告全部空间；
5. sssthresh 初始值设置为课程允许的最大接收窗口，基础 Reno 的初始 cwnd 按课程平台发布的 RFC 5681 兼容配置确定；
6. 主动关闭方的 TIME-WAIT 持续 $2 \times \text{TJU_MSL}$ ；TJU_MSL、连接重试上限和自动测试超时由课程平台统一发布；
7. 功能和性能测试文件长度应不少于 100 MB；
8. 网络环境参数、超时上限和测试数据以课程平台发布的当前测试配置为准；
9. 学生不得通过降低测试数据规模或删除异常网络条件规避规定测试。

8. 实践环境

课程提供经过验证的统一 Linux 运行环境。具体环境可以采用虚拟机、容器或教师发布的其他形式，但必须保证本地测试和线上验收环境的一致性。

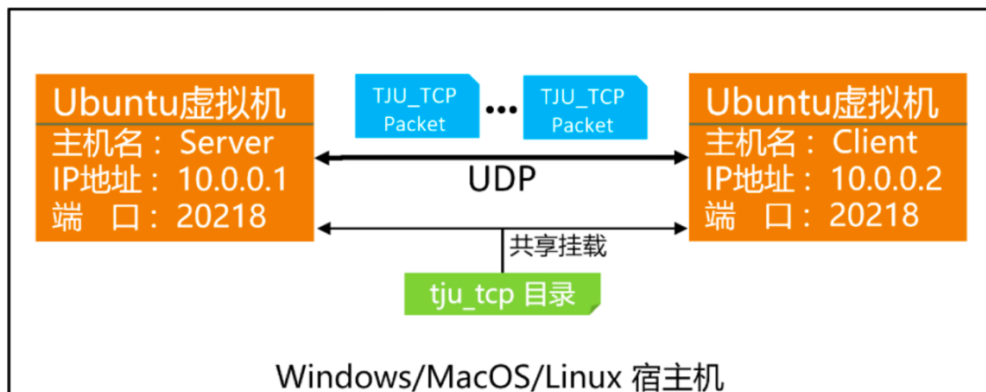


图 1 实践环境

如图 1 所示，本次实践采用 Linux 操作系统，设置两个虚拟机模拟服务端和客户端，进行双方的通信。

1. 两台虚拟机均为 Ubuntu 20.04 服务器版本, 没有 GUI。
2. 宿主机的 tju_tcp 目录同时挂载在了两台虚拟机中的 /vagrant/tju_tcp 目录下，这意味着在宿主机中对 tju_tcp 目录的任何改动都会实时同步到两台虚拟机中。

3. 建议在宿主机中使用 IDE 进行编码，在虚拟机中进行编译和测试
4. 两台虚拟机都开启了 ssh 密码验证, 并且登录用户名和密码都为 `vagrant`, `server` 虚拟机 ssh 的端口为 2200, `client` 的端口为 2222。
5. 两台虚拟机在同一个子网中, 是互通的。`server` 的 IP 地址为 10.0.0.1, `client` 的 IP 地址为 10.0.0.2。
6. 两台虚拟机之间网络通讯的默认环境设置为: 通信延迟 20ms, 最大带宽 100Mbps, 没有丢包。网络环境可以在实验过程中修改。
7. 两台虚拟机之间使用 UDP 进行数据收发, 同学们需要在 UDP 之上封装形成 `TJU_TCP`。
8. 使用 `tcconfig`、`tc/netem` 或课程指定工具配置时延、丢包率和带宽;
9. 使用 `Wireshark`、`tcpdump`、日志和 `trace` 工具观察协议行为;

课程应同时发布环境搭建、基线运行、本地测试、线上提交和常见问题说明。学生须保留实际使用的软件版本和配置, 不得仅以“在本机运行通过”代替可复现的环境记录。

有关实践环境的搭建和测试方法, 请参照:



环境搭建和测试.pdf

9. AI 辅助开发要求

9.1 必须体现 AI 参与的环节

每名学生应在以下五类环节中各保留至少一项有实质意义的 AI 协作记录:

1. 已有代码框架分析;
2. 协议或模块设计;
3. 分阶段编程实现;
4. 测试设计、日志分析或错误诊断;
5. 技术文档撰写与一致性检查。

不以 AI 对话次数、提示词长度或 AI 生成代码量作为评分依据。

9.2 AI 输出的验证

对 AI 给出的关键结论或代码, 学生至少采用一种方式进行验证:

1. 定位到具体源文件、函数、变量或标准条款;
2. 完成编译并检查警告;

3. 执行单元测试、阶段测试或边界测试；
4. 分析运行日志、**trace** 或抓包结果；
5. 与 RFC 和本说明书逐项核对；
6. 进行代码审查、并发安全检查或内存检查；
7. 对比修改前后的功能和性能结果。

学生必须能够解释所有提交代码的作用、调用关系和设计依据，并能够根据答辩要求完成局部修改或补充测试。

9.3 AI 使用记录

AI 使用记录至少包含以下内容：

类型	内容
所在阶段	框架分析、设计、实现、测试或报告
工具信息	AI 工具名称及可获得的版本或模型信息
使用目的	希望 AI 协助解决的问题
提示与上下文摘要	说明向 AI 提供了哪些必要信息，无需粘贴全部对话
AI 建议摘要	AI 给出的主要结论或代码建议
人工处理	采用、修改或拒绝了哪些内容及原因
验证方法	代码定位、标准核对、编译、测试、日志或抓包等
关联证据	Git 提交、测试记录、图表或报告章节

9.4 数据安全与禁止行为

1. 优先使用学校提供的 AI 工具；
2. 不得向公共 AI 服务上传完整课程框架、隐藏测试、教师参考实现、账号、密钥或学生个人信息；
3. 使用外部 AI 服务时，只提供完成当前问题所必需且已经脱敏的代码片段；
4. 不得提交未经理解和验证的 AI 生成代码或文字；
5. 不得使用 AI 伪造实验数据、运行日志、参考文献、Git 记录或测试结果；
6. 不得利用 AI 修改或规避验收程序；
7. 不得将其他学生的代码或报告交给 AI 改写后作为本人作品；
8. AI 使用说明不完整、刻意隐瞒重要 AI 生成内容或者不能解释提交内容，将影响过程评价和答辩成绩。

10. 验收要求及评分细则

10.1 阶段验收通用要求

三个阶段任务分别在第 1 周、第 2 周和第 3 周周末提交材料。每次提交覆盖该阶段全部子任务；具体截止时间以课程平台通知为准。

每次阶段提交应包含：

1. 对应阶段的源代码和 Git 版本标识；
2. 累积更新的进度报告，包括：
 - (1) 本地测试命令、环境参数和测试结果；
 - (2) 日志、抓包、trace 或图表；
 - (3) 本阶段代表性 AI 协作与人工验证记录；
3. 未解决问题和下一阶段计划；第三阶段提交时改为总结和后续改进建议。

自动验收平台的最终结果是功能评分的重要依据。本地测试用于发现问题，但不能替代线上验收。

10.2 代码架构分析与设计验收

重点检查：

1. 架构分析是否覆盖主要模块、接口、调用关系、数据流和修改边界；
2. 结论是否能够定位到代码或运行证据；
3. 是否识别并修正 AI 的不准确结论；
4. 设计是否覆盖全部必做需求；
5. 状态机、数据结构和测试条件是否一致；
6. 是否建立“需求—标准—设计—测试”追踪关系。

10.3 连接建立验收

建立连接的验收满分为 100 分，验收详情如下：

验收环境							
丢包率	<2%	延迟	<50ms	带宽	100Mbps	限时	5s
评分细则							
能够正确发送 SYN							20 分
能够正确响应 SYN 发送 SYNACK							40 分
能够正确响应 SYNACK，发送 ACK							40 分
具体流程							

1、测试学生的客户端

- 1) 开启自动测试程序的服务端和客户端。
- 2) 客户端调用待测试的函数 `tju_connect`。
- 3) 服务端检验能否收到 SYN 以及 SYN 的各项字段，若检测通过，则发送正确的 SYNACK 至客户端，等待客户端的 ACK。
- 4) 服务端检测能否收到 ACK 以及 ACK 的各项字段。
- 5) 关闭服务端和客户端。

2、测试学生的服务端

- 1) 开启服务端和客户端。
- 2) 服务端调用 `tju_bind`、`tju_listen` 和 `tju_accept`。
- 3) 客户端向正确的 ip 和端口发送正确的 SYN，等待 SYNACK。
- 4) 客户端检验能否收到 SYNACK 以及 SYNACK 的各项字段。
- 5) 关闭服务端和客户端。

10.4 连接关闭验收

连接关闭测试覆盖双方先后关闭和双方同时关闭两类情况，并分别测试客户端和服务端功能。验收同时检查 CLOSE 之前提交的数据是否排列在 FIN 之前并得到可靠传输，以及主动关闭方是否按课程参数进入 TIME-WAIT。

断开连接的验收满分为 100 分，断开连接将测试双方先后断开和双方同时断开两种情况，验收详情如下：

验收环境							
丢包率	0%	延迟	<50ms	带宽	100Mbps	限时	27s
评分细则-双方先后断开连接-测试客户端功能							
能够发送正确的 FINACK（第一次挥手）							15 分
能响应正确的 FINACK，发送正确的 ACK（第四次挥手）							15 分
评分细则-双方同时断开连接-测试客户端功能							
能够在已发送 FINACK 还未收到 ACK 时正确响应对方的 FINACK							10 分
能够响应 FINACK 并发送正确 ACK							10 分
评分细则-双方先后断开连接-测试服务端功能							
能够响应 FINACK 并发出正确 ACK（第二次挥手）							15 分
能发送正确 FINACK（第三次挥手）							15 分
评分细则-双方同时断开连接-测试服务端功能							
能够在已发送 FINACK 还未收到 ACK 时正确响应对方的 FINACK							10 分
能够响应 FINACK 并发送正确 ACK							10 分
具体流程-双方先后断开连接-测试客户端功能							
1) 开启自动测试程序第服务端和客户端，双方首先建立连接							

2) 模拟端调用待测试的 tju_close。 3) 服务端检查 FINACK，响应发送 ACK。 4) 服务端发送 FINACK，等待最后的 ACK。 5) 服务端检查最后的 ACK。 6) 重新运行服务端和客户端，双方建立连接。 7) 这次服务端发送 FINACK，等待客户端的 ACK。 8) 服务端检查 ACK。
具体流程-双方同时断开连接-测试客户端功能
1) 开启自动测试程序第服务端和客户端，双方首先建立连接。 2) 客户端调用待测试的 tju_close。 3) 服务端检查 FINACK，首先发送自己的 FINACK。 4) 服务端发送 FINACK 后再响应客户端，发送 ACK。 5) 服务端等待并检查最后的 ACK。
具体流程-双方先后断开连接-测试服务端功能
1) 开启自动测试程序的服务端和客户端，双方首先建立连接 2) 模拟端调用待测试的 tju_close。 3) 客户端发送 FIN，等待 ACK。 4) 客户端检查 ACK，等待 FIN 或 FINACK。 5) 客户端检查 FIN/FINACK，发送 ACK。
具体流程-双方同时断开连接-测试服务端功能
1) 开启自动测试程序第服务端和客户端，双方首先建立连接。 2) 服务端调用待测试的 tju_close。 3) 客户端发送 FIN。 4) 客户端检查 FIN 或 FINACK 后再响应客户端，发送 ACK。 5) 服务端等待并检查最后的 ACK。

10.5 可靠数据传输验收

可靠数据传输的验收满分为 100 分，验收详情如下：

验收环境							
丢包率	<10%	延迟	<300ms	带宽	100Mbps	限时	60s
评分细则							
服务端连续正确接收的来自客户端的数据							
具体流程							
1) 开启自动测试程序的服务端和客户端。 2) 服务端调用待测试的 tju_bind、tju_listen 和 tju_accept。 3) 客户端调用待测试的 tju_connect。 4) 建立连接后，客户端读取总大小为 100MB 的测试文件（test.txt），发送完毕调用 tju_close。 5) 服务端将收到的数据写入接收文件(recv.txt)。							

注意：

- 1) 可靠数据传输要首先建立连接，因此建立连接的过程也存在丢包，需要做相应处理。
- 2) 客户端需要调用 `tju_close`，在 TCP 连接关闭时如果发送缓冲区还存有数据，需要待数据发送完毕后再关闭连接。

10.6 流量控制、基础拥塞控制与挑战任务验收

流量控制和拥塞控制的部分目前不能进行自动测试，为此提供了 `trace` 文件格式和绘图脚本。

同学们需要将绘图结果体现在报告中并加以解释。运行程序后生成 `trace` 文件，通过提供的绘图脚本绘制序列号、发送窗口大小等信息随时间变化的图像，进而展示流量控制和拥塞控制的过程。

注意，**不建议使用全自动的测试脚本进行抓包和绘图**，同学们最好采用手动方式调整网络环境，进行抓包和绘图，以保持最大的灵活性，获得符合要求的图像。

流量控制和拥塞控制的脚本和测试方法，请参考：



流量控制和拥塞控制测试说明.pdf

流量控制必做验收重点包括：

1. 通告窗口是否反映接收缓冲区可用空间；
2. 16 位通告窗口是否正确限制在 0 至 65535 字节，且不存在整数截断引起的回绕；
3. 发送窗口是否受通告窗口约束；
4. 零窗口及窗口恢复是否得到正确处理；
5. 零窗口探测是否按 RTO 和指数退避发送并得到响应；
6. 发送端和接收端是否具备基本 SWS 避免行为。

基础 Reno 必做验收重点包括：

1. 实际在途数据量是否同时受 `rwnd` 和 `cwnd` 约束；
2. `cwnd` 在慢启动阶段是否按 ACK 快速增长，在拥塞避免阶段是否约每个 RTT 增长一个 SMSS；
3. RTO 超时后 `ssthresh` 是否根据 `FlightSize` 降低，`cwnd` 是否降低到不超过一个 SMSS 并重新慢启动；
4. 三次重复 ACK 后是否执行基本快速重传并降低窗口；
5. `cwnd`、`ssthresh`、`rwnd` 和在途数据量之间的关系是否正确。

挑战任务根据所选方向分别检查完整快速恢复、部分确认处理、SACK 记分板、RACK-TLP 时间判定或 CUBIC 窗口函数是否符合相应 RFC，并检查其与基础 Reno 的可复现对比结果。

所有验收均检查：

1. 图表是否能够由提交的原始 trace 和脚本复现。

10.7 性能测试

至少选取丢包率、时延、带宽、发送/接收缓冲区大小和数据报文长度中的两类变量开展对照实验。性能指标至少包括吞吐率，也可以增加完成时间、重传次数、有效数据比例等指标。

学生应：

1. 说明实验问题和变量控制方法；
2. 记录完整的环境与参数；
3. 重复实验并说明数据处理方法；
4. 提交原始数据和绘图脚本；
5. 分析结果形成的原因；
6. 说明协议设计或实现的局限性及可能的改进方向。

11. 测试帮助文档汇总

1. 有关实践环境的搭建和测试方法，请参照：



环境搭建和测试.pdf

2. 有关本地自动测试的具体步骤，请参考：



本地测试说明.pdf

3. 有关如何提交线上验收网站，查看自己的验收成绩和排行榜，请参考：



提交线上验收说明.pdf

4. 流量控制和拥塞控制的脚本和测试方法，请参考：



流量控制和拥塞控制测试说明.pdf

5. 为方便同学开发，整理了往年实践中同学们频繁遇到的问题，并针对一些难点进行提示，形成了如下文档：



频繁遇到的问题和提示.pdf

12. 提交、独立性与学术诚信

1. 每名学生使用独立仓库、独立报告、独立实验数据和独立答辩材料；
2. 学生之间可以讨论 TCP 原理、RFC 理解、通用工具使用和调试方法，但不得交换源代码、完整设计文档、AI 生成结果、测试数据或报告；
3. 所有图表必须能够由本人提交的代码、原始数据和脚本复现；
4. 教师提供的基础框架和测试代码在查重时应作为公共代码排除，重点检查学生新增和修改部分；
5. 抄袭、伪造数据、代做、绕过测试、冒用他人成果等行为按学校相关规定处理；
6. 课程源码在课程结束后三年内不得公开，不得在学生之间流传，也不得上传至公开代码仓库或未经批准的 AI 服务；如学校另有规定，以学校规定为准；
7. 学生对本人提交的全部内容承担最终责任，不能以“由 AI 生成”为理由免除责任。

13. 参考资料

1. J. Kurose and K. Ross, *Computer Networking: A Top-Down Approach*, 8th Edition, Pearson, 2021.
2. RFC 9293, *Transmission Control Protocol (TCP)*, <https://www.rfc-editor.org/rfc/rfc9293.html>.
3. RFC 6298, *Computing TCP's Retransmission Timer*, <https://www.rfc-editor.org/rfc/rfc6298.html>.
4. RFC 5681, *TCP Congestion Control*, <https://www.rfc-editor.org/rfc/rfc5681.html>.
5. RFC 2018, *TCP Selective Acknowledgment Options*, <https://www.rfc-editor.org/rfc/rfc2018.html>.
6. RFC 6582, *The NewReno Modification to TCP's Fast Recovery Algorithm*, <https://www.rfc-editor.org/rfc/rfc6582.html>.
7. RFC 6675, *A Conservative Loss Recovery Algorithm Based on Selective Acknowledgment (SACK) for TCP*, <https://www.rfc-editor.org/rfc/rfc6675.html>.
8. RFC 7323, *TCP Extensions for High Performance*, <https://www.rfc-editor.org/rfc/rfc7323.html>.
9. RFC 8985, *The RACK-TLP Loss Detection Algorithm for TCP*, <https://www.rfc-editor.org/rfc/rfc8985.html>.
10. RFC 9438, *CUBIC for Fast and Long-Distance Networks*, <https://www.rfc-editor.org/rfc/rfc9438.html>.
11. RFC 793, *Transmission Control Protocol*, <https://www.rfc-editor.org/rfc/rfc793.html>.
12. W. Richard Stevens, *TCP/IP Illustrated, Volume 1: The Protocols*.
13. Gary R. Wright and W. Richard Stevens, *TCP/IP Illustrated, Volume 2: The Implementation*.
14. TCConfig, <https://github.com/thombashi/tcconfig>.